La pile

Introduction

- La pile est une structure de stockage de données particulière
- Elle fait partie de la mémoire (segment particulier) et sert à stocker temporairement des informations
- Nous allons aborder comment elle est organisée et comment elle fonctionne

Principe de fonctionnement

- Les données sont stockées dans la pile par empilement, c'est à dire qu'elles sont déposées en sommet de pile
- On ne peut empiler qu'un élément à la fois
- Si la pile possède plusieurs éléments, ils ne pourront être retirés par désempilement dans l'ordre inverse de l'empilage
- Une pile a un fonctionnement de type LIFO (Last In First Out): le dernier élément empilé et le premier que l'on peut retirer
- Pour accéder à un élément dans la pile il faut obligatoirement désempiler tous les éléments qui se trouvent au dessus

Principe de fonctionnement

- La pile est définie dans un segment de mémoire particulier: le STACK SEGMENT
- Elle est constituée d'un ensemble d'octets se trouvant dans le segment dont l'adresse est stockée dans le registre SS (Stack Segment)
- Le processeur suit le processus d'empilement et de désempilement grâce au registre SP (Stack Pointer)
- Le registre SP contient le déplacement (offset) dans le segment de pile pour accéder au dernier élément empilé
- L'adresse logique de l'élément en sommet de pile (dernier élément empilé) est SS:SP

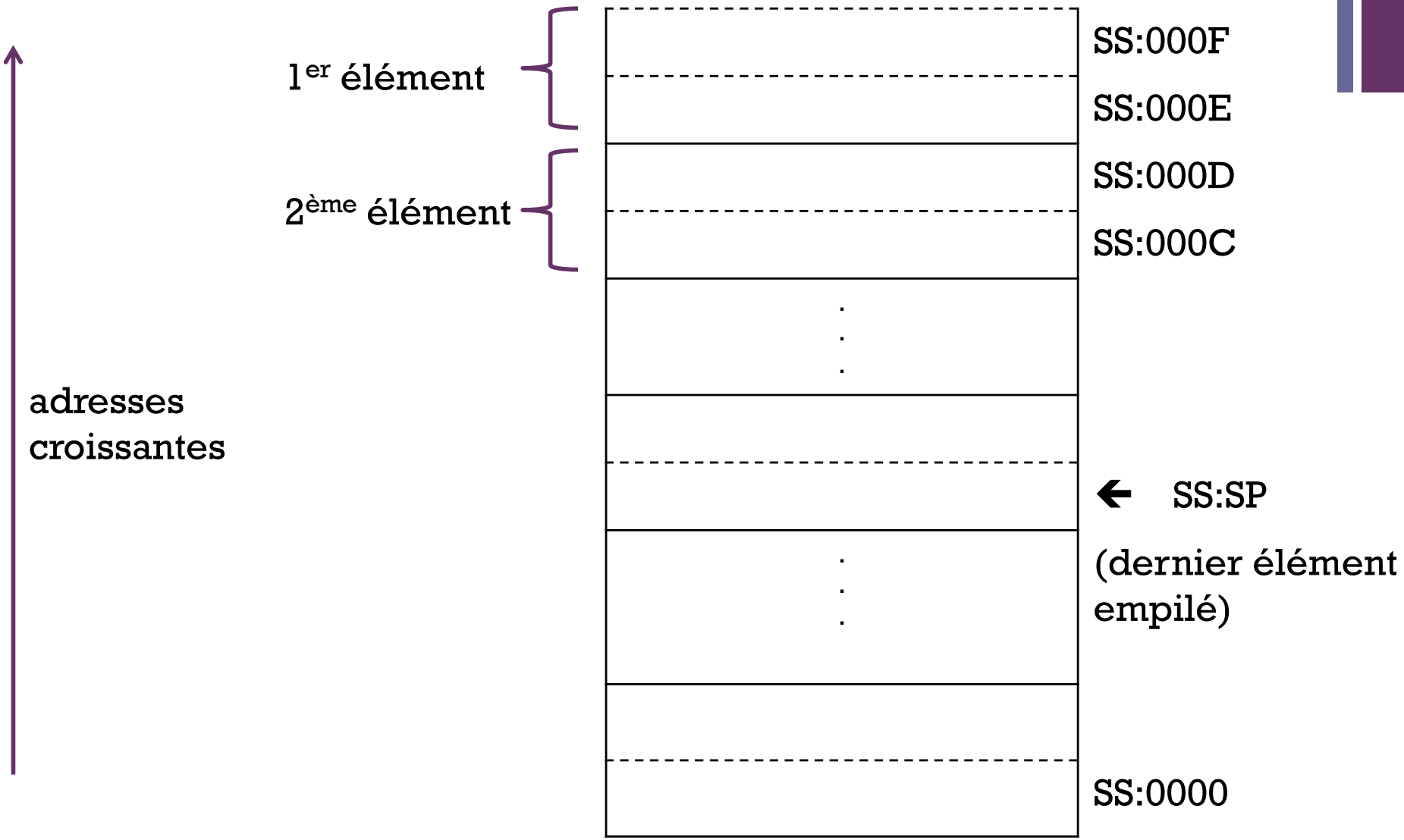
Principe de fonctionnement

- L'adresse SS:0000 pointe sur le premier octet de la zone mémoire définie pour la pile
- La taille de chaque élément empilé est de 2 octets

Gestion de la pile

- La taille de la zone mémoire réservée pour la pile est variable d'un programme à l'autre et est définie par le programmeur
- Cette zone mémoire à la particularité d'être gérée à l'envers: son remplissage s'effectue depuis la fin de la zone mémoire de la pile
- Les premiers éléments empilés se trouvent donc à une adresse mémoire plus grande que les derniers éléments empilés

Gestion de la pile



Déclaration de la pile

- Pour pouvoir être utilisée dans un programme le segment de pile doit d'abord être déclaré
- Il faut réserver un espace mémoire suffisant, et initialiser les registres SS et SP
- Exemple de déclaration:

```
seg_pile     SEGMENT stack  ; mot clé de déclaration
             DW 100 dup(?)  ; reserve 100 mots non- initialisés
base_pile    EQU  this word ; repère l'adresse du dernier élement de la pile +1
             ENDS

dans le code, on aura

             ASSUME CS:code, DS:data, SS:seg_pile
             ...
             MOV AX,seg_pile;
             MOV SS,AX;
             LEA SP,base_pile;
```

Les instructions de manipulation de la pile

- Les deux seules opérations autorisées sur une pile sont les instructions d'empilage PUSH et de désempilage POP
- Ces deux instructions prennent pour opérande un registre ou une adresse mémoire sur 16 bits
- PUSH registre
  PUSH mem : empile le contenu du registre ou de la mémoire dans la pile
- POP registre
  POP mem : désempile le contenu en sommet de pile et le place dans le registre ou l'adresse mémoire spécifiée

Les instructions de manipulation de la pile

- Exemple: stockage temporaire du contenu de registres

```
PUSH AX       ; empilement du contenu de AX
PUSH BX       ; empilement du contenu de BX
MOV BX,truc ; truc <-- truc + AX
ADD AX,BX     ;
MOV truc,AX
POP BX        ; récupération de BX
POP AX        ; récupération de AX
```

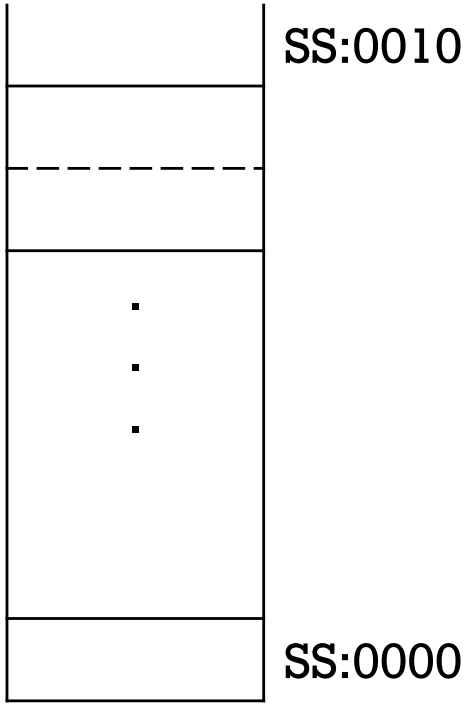
Action des instructions PUSH et POP

- Lorsqu'une instruction PUSH est rencontrée dans le programme par le processeur, SP est d'abord décrémenté de 2, puis le contenu du registre ou de la zone mémoire spécifiée est copié à l'adresse correspondant à SS:SP
- Dans le cas d'une instruction POP, le registre reçoit le contenu se trouvant à l'adresse indiquée par SS:SP puis la valeur de SP est incrémentée de 2
- L'exécution de l'instruction POP n'efface pas le contenu de la case mémoire pointée originalement avant l'incrémentation par SS:SP, mais à l'exécution du PUSH suivant le contenu de cette case de la pile sera écrasée par la nouvelle valeur empilée

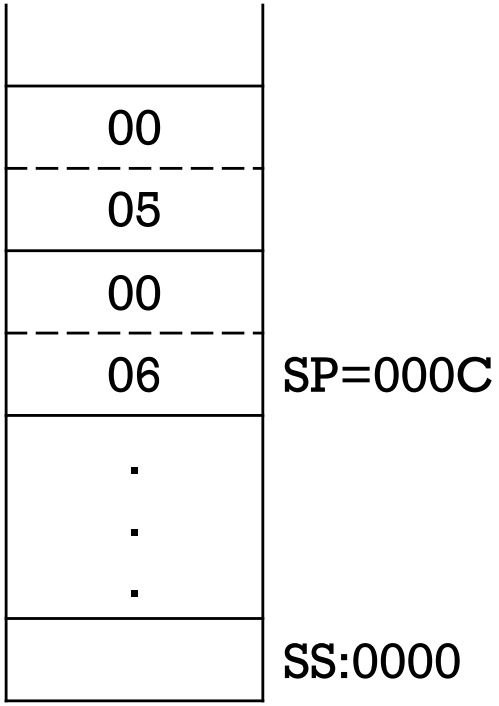
Action des instructions PUSH et POP

- Exemple: on dispose d'une pile de 16 octets référencés par SS:0000, et à l'initialisation SS:SP vaut SS:0010

```
MOV AX, 0005h;
MOV BX, 0006h;
PUSH AX       ;
PUSH BX       ;
MOV BX,0003h  ;
POP BX        ;
```



état initial

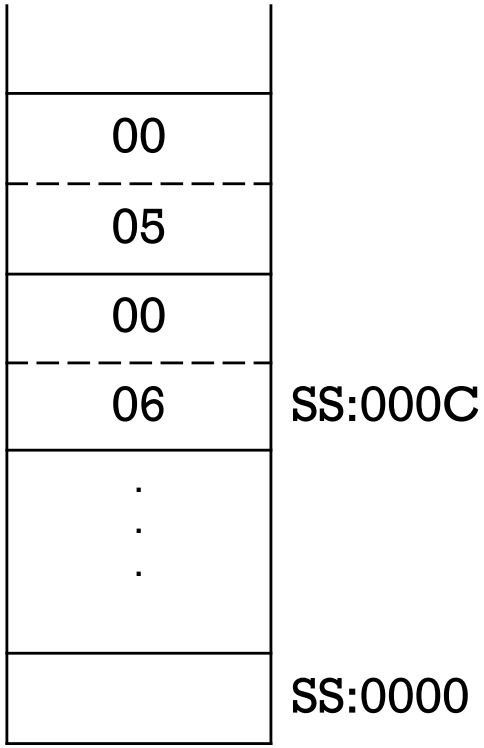


après PUSH BX

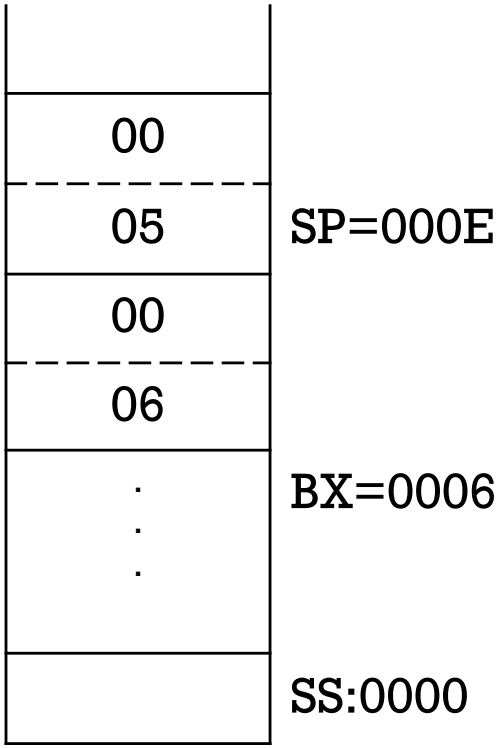
Action des instructions PUSH et POP

- Exemple: on dispose d'une pile de 16 octets référencés par SS:0000, et à l'initialisation SS:SP vaut SS:0010

```
MOV AX, 0005h;
MOV BX, 0006h;
PUSH AX       ;
PUSH BX       ;
MOV BX,0003h  ;
POP BX        ;
```



après PUSH BX



Après POP BX

Autres instructions de gestion de la pile

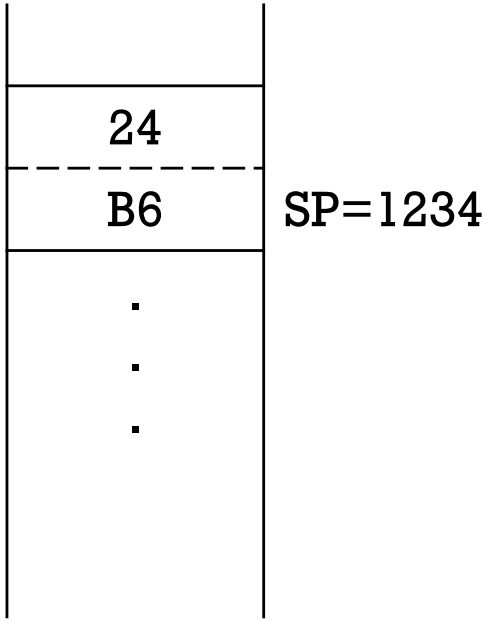
- Les instructions PUSHA et POPA permettent d'empiler et de désempiler les registres AX, BX, CX, DX, SP, BP, SI et DI (***attention: seulement disponibles à partir du microprocesseur 80286***)

- Les instructions PUSHF et POPF permettent d'empiler de désempiler le registre d'état de l'UAL

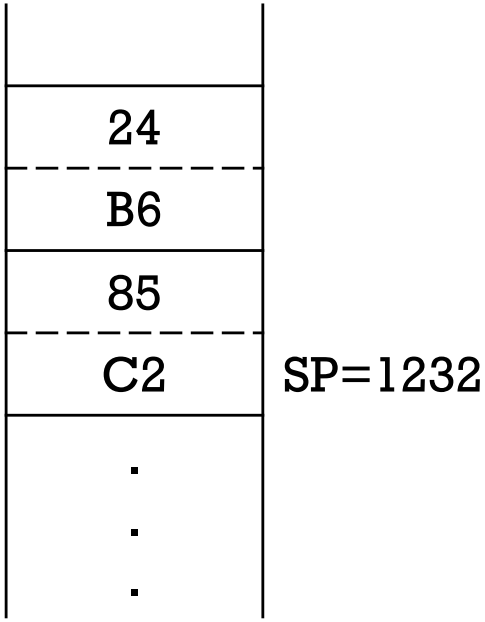
Utilisation de la pile (exemple)

- SP=1236h, AX=24B6h, DI=85C2, DX=5F93h

```
PUSH AX
PUSH DI
PUSH DX
POP CX
POP DX
POP BX
```

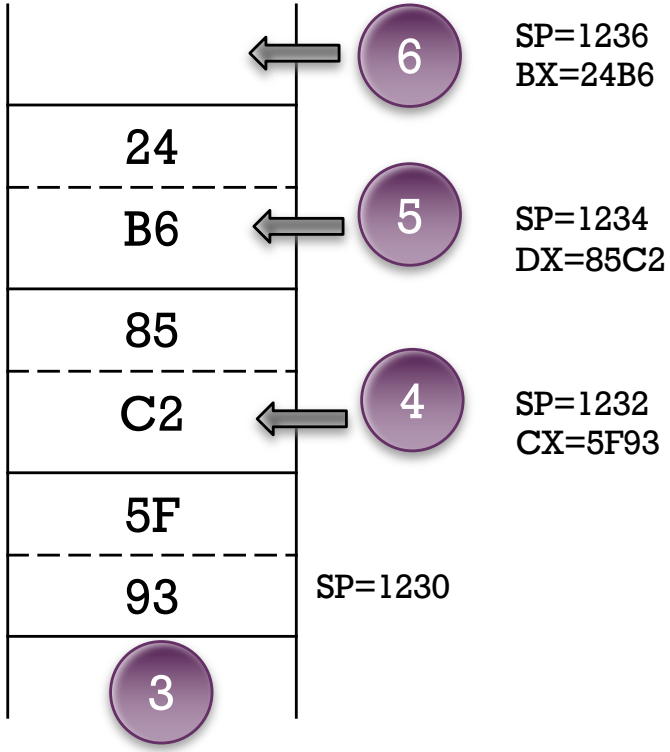


1



2

Utilisation de la pile (exemple)

- SP=1236h, AX=24B6h, DI=85C2, DX=5F93h

```
PUSH AX
PUSH DI
PUSH DX
POP CX
POP DX
POP BX
```